GROUPD – A CLOUD-BASED AI-POWERED CAMPUS TEAM-MATCHING PLATFORM

CLOUD COMPUTING

Submitted by

ATHIENA RACHEL (230701046)
ATHIRA D R (230701047)
NITHYASHREE M (230701221)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
RAJALAKSHMI ENGINEERING COLLEGE

MAY 2026
ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this project report titled **"Groupd – A Cloud-Based AI-Powered Campus Team-Matching Platform"** is the bonafide work of **ATHIENA RACHEL (230701046), ATHIRA D R (230701047), NITHYASHREE M (230701221)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

_

SIGNATURE

[Name, Designation]

Head of The Department

Department of Computer Science and Engineering

Rajalakshmi Engineering College College

SIGNATURE

[Name, Designation]

Supervisor / Guide

Department of Computer Science and Engineering

Rajalakshmi Engineering College

Submitted to Project Viva Voce Examination held on _____

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering blessings through this endeavour. Our sincere thanks to our Chairman Mr. S. MEGANATHAN, B.E, F.I.E., our Vice Chairman Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S., and our respected Chairperson Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D., for providing us with the requisite infrastructure at their premier institution.

Our sincere thanks to Dr. S.N. MURUGESAN, M.E., Ph.D., our beloved Principal for his kind support. We express our sincere thanks to the Head of the Department of Computer Science and Engineering for guidance and encouragement throughout the project work. We convey our deepest gratitude to our internal guide for valuable guidance throughout the course of this project.

We would also like to thank all faculty members and our peers for their continuous support and feedback during the development of Groupd.

ABSTRACT

Students on campus often struggle to form effective project teams due to the absence of a centralized and intelligent platform that facilitates skill-based teammate discovery. Traditional approaches such as social media posts, group chats, and word-of-mouth referrals are time-consuming, informal, and lack transparency regarding skills, availability, and complementarity. Studies show that poorly matched teams result in reduced collaboration quality, delayed project completion, and diminished learning outcomes. Existing platforms focus on large-scale professional networking rather than addressing the intra-campus team formation problem.

The proposed project, Groupd, is a secure and scalable cloud-based AI-powered campus team-matching platform that enables students to create skill profiles, discover teammates through semantic AI matching, create and join project teams, and communicate via real-time messaging. The platform uses Azure OpenAI embeddings and Azure AI Search for intelligent skill-to-skill matching, ensuring students are connected with collaborators whose strengths complement their own.

The system architecture is designed using a cloud-native, service-oriented approach deployed entirely on Microsoft Azure. The React/TypeScript frontend is hosted on Azure Static Web Apps, while the Python Flask backend is deployed via Azure Container Apps (containerized with Docker). Azure Cosmos DB (NoSQL, serverless) handles all data persistence, and Azure Blob Storage manages media uploads. Authentication is implemented using JWT-based institutional email verification. CI/CD pipelines automate both frontend and backend deployment through GitHub Actions.

The expected outcome is a reliable, intelligent, and user-friendly platform that reduces the time and friction involved in campus team formation, improves project collaboration quality, and provides students with AI-powered teammate recommendations tailored to their specific project needs.

TABLE OF CONTENTS

CHAPTER 1 - INTRODUCTION

1.1 PROBLEM STATEMENT

Students in engineering colleges frequently face difficulty in forming well-balanced project teams. The absence of a centralized, intelligent campus collaboration platform forces students to rely on fragmented and inefficient methods:

- Difficulty in Finding Complementary Teammates: Students cannot easily identify peers with specific technical skills such as machine learning, frontend development, or database management. Over 60% of student teams report skill imbalance as a primary reason for project underperformance.
- Lack of Verified Skill Profiles: Existing informal methods provide no mechanism for verifying or showcasing technical skills, GitHub contributions, or project experience. This leads to mismatches and reduced team effectiveness.
- Inefficient Team Discovery: Students rely on WhatsApp groups, class announcements, or personal networks for team formation. This excludes students from other departments or batches who may be ideal collaborators.
- No Intelligent Matching: Current methods provide no AI-powered compatibility scoring. Teams are formed based on proximity or social connections rather than skill complementarity.
- Scalability and Coordination Challenges: As the number of students, projects, and hackathons grows, manually coordinating team formation becomes increasingly unsustainable without an automated, cloud-based platform.

These issues collectively result in poor team composition, wasted collaboration potential, and reduced quality of student projects.

1.2 OBJECTIVE OF THE PROJECT

The primary objective of this project is to design and develop a cloud-based, AI-powered campus team-matching platform that enables students to create detailed skill profiles, discover and connect with compatible teammates, and collaborate on academic and hackathon projects efficiently.

1. Enable Intelligent Skill-Based Team Matching

Implement AI-powered semantic matching using Azure OpenAI embeddings and Azure AI Search so that students are matched with teammates whose skills and interests complement their own project requirements.

2. Provide Secure Cloud-Based User Authentication

Implement JWT-based authentication with institutional email domain restriction, ensuring only verified campus students can access the platform. Azure AD B2C integration provides enterprise-grade identity management.

3. Enable Real-Time Collaboration Features

Integrate Azure Web PubSub for real-time messaging and Azure Communication Services for notifications, enabling students to communicate directly within the platform.

4. Deploy a Scalable Cloud-Native Architecture

Utilize Azure Container Apps for backend deployment, Azure Static Web Apps for frontend hosting, Azure Cosmos DB for serverless data storage, and Azure Blob Storage for media management — ensuring scalability, high availability, and cost efficiency.

5. Automate Deployment via CI/CD Pipelines

Implement GitHub Actions workflows to automate frontend and backend builds, Docker image pushes to Azure Container Registry, and deployment to Azure cloud services on every code commit.

1.3 SCOPE AND BOUNDARIES

The scope of this project encompasses the complete development and deployment of a cloud-native campus team-matching web application integrated with AI-powered semantic matching, real-time communication features, and automated CI/CD pipelines.

The system supports core platform functionalities including student registration and login with institutional email validation, skill profile creation and management, team creation and discovery, AI-powered match scoring, real-time messaging, and notification delivery. The platform is deployed entirely on Microsoft Azure using a containerized microservice-oriented architecture.

Features outside the current scope include mobile application development, payment integrations, video calling within the platform, and multi-institution federation. These are reserved for future enhancement phases.

1.4 STAKEHOLDERS AND END USERS

Stakeholders:

- Project Developers / Team Members: Responsible for designing, developing, deploying, and maintaining the frontend, backend APIs, AI workflows, database systems, and cloud infrastructure.
- Platform Administrators: Manage user accounts, review reported content, and maintain platform integrity through an admin dashboard.

End Users:

- Students: Students who want to create skill profiles, discover compatible teammates, create or join project teams, and communicate with potential collaborators.
- Team Leads / Project Creators: Students who post new team openings, define required skill sets, and review match candidates.
- Administrators: Authorized personnel responsible for managing user verification, platform moderation, and system health monitoring.

1.5 TECHNOLOGIES USED

The project incorporates a modern technology stack tailored for performance, scalability, and AI-powered matching:

Table 1. Frontend Technologies

Technology	Purpose
React / TypeScript	Component-based SPA frontend
Tailwind CSS	Utility-first responsive UI styling
React Router v6	Client-side routing and protected routes
Axios	HTTP client for API communication
Vite	Fast build tooling and dev server

Table 2. Backend Technologies

Technology	Purpose
Python / Flask	REST API backend framework
PyJWT	JWT token generation and validation
Gunicorn	WSGI HTTP server for production
Azure Cosmos DB SDK	NoSQL database client
Azure Storage SDK	Blob storage for media uploads

Table 3. Cloud and Infrastructure

Service	Purpose
Azure Static Web Apps	Frontend hosting with integrated CI/CD
Azure Container Apps	Containerized backend deployment with auto-scaling
Azure Container Registry	Docker image registry (groupdregistry)
Azure Cosmos DB (NoSQL)	Serverless document database (Southeast Asia)
Azure Blob Storage	Media and profile image storage
Azure AD B2C	Identity management and institutional MFA
Azure Front Door	CDN, SSL termination, DDoS protection

Table 4. AI and Machine Learning

Service	Purpose
Azure OpenAI (Embeddings)	Generate semantic skill vectors for matching
Azure AI Search	Semantic similarity search across student profiles
Azure Functions	Triggered by Cosmos DB change feed to generate embeddings

Table 5. DevOps and CI/CD

Tool	Purpose
GitHub Actions	Automated CI/CD pipelines for frontend and backend
Docker	Containerization of Flask backend
Azure Container Registry	Storage and versioning of Docker images
Azure Web PubSub	Real-time messaging infrastructure
Azure Communication Services	Email and push notification delivery

1.6 ORGANIZATION OF THE REPORT

This report is organized to provide a systematic and comprehensive overview of the entire project development lifecycle.

Chapter 1 introduces the project background, problem statement, objectives, scope, stakeholders, technologies used, and overall report structure.

Chapter 2 presents the system architecture and design, including frontend-backend interaction, Azure cloud integration, Cosmos DB structure, authentication workflow, and AI service architecture.

Chapter 3 discusses the DevOps implementation including Docker containerization, GitHub Actions CI/CD pipelines, Azure Container Apps deployment, and AI integration workflows.

Chapter 4 explains cloud operations and security including JWT authentication, Azure AD B2C, CORS configuration, monitoring via Azure Application Insights, and API security.

Chapter 5 presents the project results, system screenshots, performance observations, challenges faced, and overall project evaluation.

Chapter 6 concludes the report with a summary of achievements, limitations, and future enhancement possibilities such as mobile application support and advanced recommendation systems.

CHAPTER 2 - SYSTEM DESIGN AND ARCHITECTURE

2.1 REQUIREMENT SUMMARY

Functional Requirements

The system consists of multiple cloud-based components deployed on Microsoft Azure to support a scalable, secure, and intelligent campus team-matching platform.

The frontend application is developed using React and TypeScript and hosted on Azure Static Web Apps. This service provides reliable hosting, integrated GitHub connectivity, and automated CI/CD deployment workflows through GitHub Actions. The frontend covers 14 pages including landing, login, register, dashboard, profile, search, teams, match, create team, team detail, messages, notifications, about, and admin pages.

The backend logic is implemented using Python Flask, containerized with Docker, and deployed via Azure Container Apps (groupdbackend). The backend exposes REST API endpoints handling user authentication, profile management, team CRUD operations, AI match retrieval, messaging, and notification workflows. CORS is configured to securely enable communication between the frontend static app and the backend container.

All user data, team information, match scores, messages, and notifications are stored in Azure Cosmos DB (NoSQL, serverless, Southeast Asia region). Cosmos DB is configured with a serverless capacity model to optimize cost during development and scale automatically under production load.

Profile images and media uploads are stored in Azure Blob Storage. Azure AD B2C provides institutional email-based identity management with MFA support. Azure Functions are triggered by the Cosmos DB change feed to automatically generate OpenAI embeddings whenever a student profile is created or updated.

Non-Functional Requirements

- Performance: API response times targeted below 300ms; frontend page load optimized via Azure CDN and Static Web Apps.
- Scalability: Azure Container Apps auto-scales backend replicas based on HTTP traffic; Cosmos DB scales throughput serverlessly.
- Availability: Azure SLA-backed services ensure 99.9%+ uptime for all deployed components.
- Security: JWT tokens with institutional domain validation, HTTPS-only via Azure Front Door, CORS restrictions on backend APIs.
- Maintainability: Modular Flask blueprint architecture; React component-based frontend with TypeScript type safety.
- Cost Efficiency: Student Azure subscription with serverless Cosmos DB and consumption-based Container Apps billing.

2.2 PROPOSED SOLUTION OVERVIEW

Groupd is a cloud-native campus collaboration platform built on a modular service-oriented architecture. At the presentation layer, the React/TypeScript frontend hosted on Azure Static Web Apps communicates with the Flask backend deployed on Azure Container Apps via REST APIs secured with JWT bearer tokens.

The backend orchestrates all business logic — user registration and authentication, profile management, team operations, real-time messaging routing, and AI match retrieval. Persistent storage is handled by Azure Cosmos DB for structured NoSQL document data and Azure Blob Storage for binary media.

The AI matching pipeline operates as follows: when a student creates or updates their skill profile, the Cosmos DB change feed triggers an Azure Function that calls Azure OpenAI to generate a semantic embedding vector for that profile. This vector is indexed in Azure AI Search. When a student requests matches, the backend queries Azure AI Search with the requesting student's embedding to retrieve the top-N semantically similar profiles, which are returned as match recommendations.

Real-time messaging is powered by Azure Web PubSub, with Azure Communication Services handling email and push notifications for team invitations and match alerts.

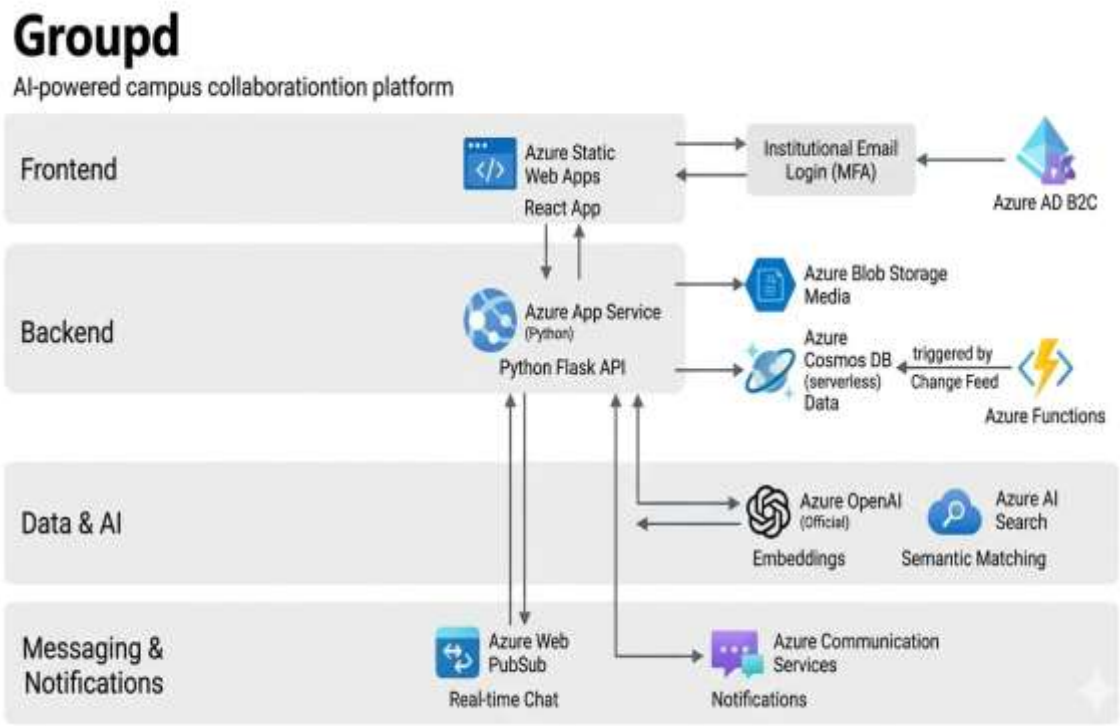

Figure 1. Groupd System Architecture Overview

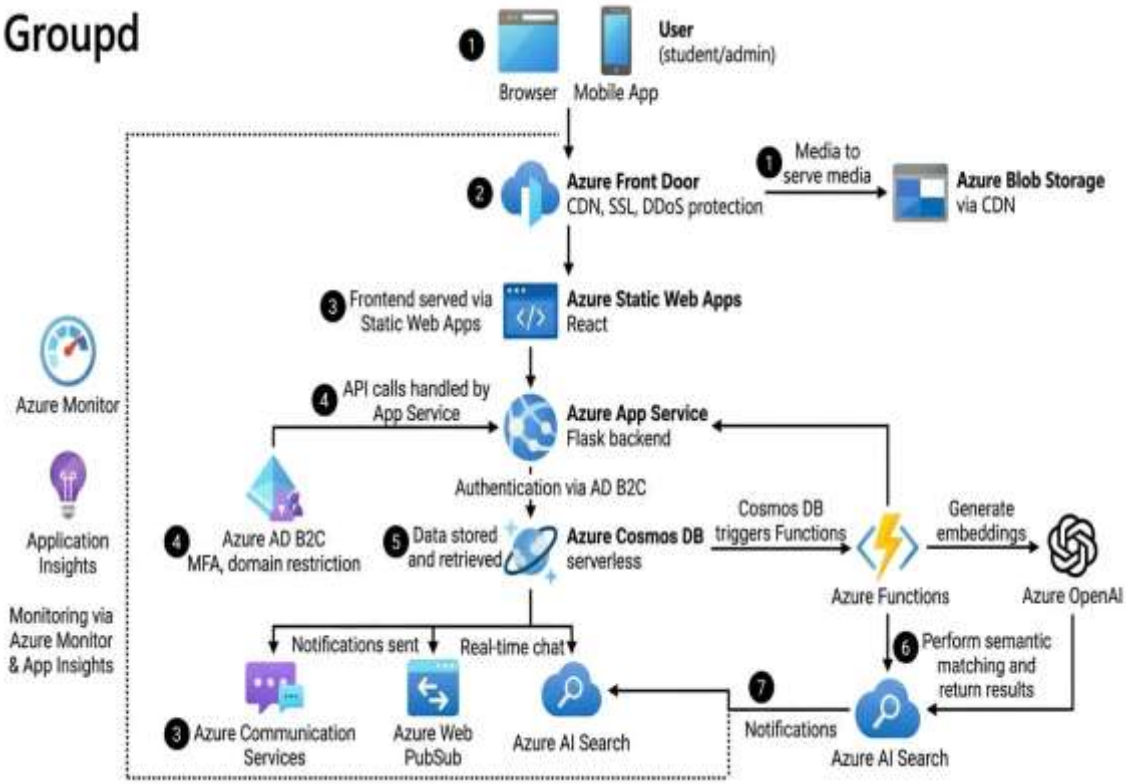

Figure 2. Groupd Cloud Deployment Flow Diagram

2.3 CLOUD DEPLOYMENT STRATEGY

The deployment strategy leverages Microsoft Azure as the primary cloud platform, utilizing a combination of Platform-as-a-Service (PaaS) and serverless services to minimize infrastructure management while ensuring scalability and reliability.

The frontend React application is deployed through Azure Static Web Apps, providing global CDN distribution, automatic HTTPS, and GitHub Actions integration for zero-touch deployments. The backend Flask application is containerized using Docker and deployed to Azure Container Apps (groupdbackend), which provides automatic ingress, HTTPS termination, and replica auto-scaling based on HTTP concurrency.

Docker images are built during the GitHub Actions CI workflow and pushed to Azure Container Registry (groupdregistry). Azure Container Apps then pulls the latest image from the registry on each deployment. Environment variables including Cosmos DB connection strings, JWT secrets, and Azure OpenAI API keys are injected as Container App secrets at runtime, ensuring no sensitive data is stored in source code.

The deployment is constrained to Azure regions available on the student subscription: Southeast Asia, East Asia, Central India, Austria East, and UAE North. Cosmos DB and Container Apps are deployed in the Southeast Asia region for optimal latency.

2.4 INFRASTRUCTURE REQUIREMENTS

Compute Resources:

- Azure Container Apps (groupdbackend): Hosts the Dockerized Flask/Gunicorn backend. Consumption plan with auto-scaling enabled.
- Azure Static Web Apps (groupd-frontend): Hosts the React/TypeScript frontend. Free tier with integrated GitHub Actions CI/CD.
- Azure Functions: Triggered by Cosmos DB change feed for automated embedding generation pipeline.

Storage Requirements:

- Azure Cosmos DB (NoSQL, Serverless, Southeast Asia): Document storage for users, teams, matches, messages, and notifications.
- Azure Blob Storage: Hot tier storage for profile images and media uploads.

AI and Messaging:

- Azure OpenAI Services: Embedding generation for semantic profile matching.
- Azure AI Search: Vector similarity search index over student profiles.
- Azure Web PubSub: Real-time WebSocket messaging between students.
- Azure Communication Services: Email notifications and push alerts.

2.5 AZURE SERVICES MAPPING AND JUSTIFICATION

Table 6. Azure Services Mapping

Requirement	Azure Service	Purpose	Justification	Est. Cost (₹)
Frontend Hosting	Azure Static Web Apps	Host React/TS app globally	Integrated CI/CD, SSL, CDN	Free (Student)
Backend APIs	Azure Container Apps	Host Flask backend container	Auto-scaling, HTTPS ingress	600
Database	Cosmos DB (NoSQL)	Store all platform data	Serverless, flexible schema	400
File Storage	Azure Blob Storage	Profile images & media	Scalable cloud storage	200
AI Matching	Azure OpenAI + AI Search	Semantic skill matching	State-of-the-art embeddings	500
Real-time Chat	Azure Web PubSub	Live messaging between users	WebSocket at scale	200
Notifications	Azure Comm. Services	Email and push alerts	Managed notification infra	100
Auth & Identity	Azure AD B2C	Institutional MFA login	Enterprise identity mgmt	Free
Container Registry	Azure ACR	Store Docker images	Tight Azure integration	150
CI/CD	GitHub Actions	Automate deployments	GitHub-native, free tier	Free

CHAPTER 3 - DEVOPS IMPLEMENTATION

3.1 CONTINUOUS INTEGRATION AND DEPLOYMENT (CI/CD) SETUP

The project uses two CI/CD pipelines implemented with GitHub Actions on the repository AthienaRae/groupd. The pipelines automate frontend deployment, backend container builds and pushes, and cloud deployment workflows, ensuring that every code commit triggers a validated and consistent deployment to Azure.

Frontend Deployment Pipeline (Azure Static Web Apps CI/CD)

The frontend pipeline triggers on every push to the main branch. It checks out the React/TypeScript source, installs Node.js dependencies, runs the Vite build process, and deploys the compiled static assets to Azure Static Web Apps. The pipeline uses the AZURE_STATIC_WEB_APPS_API_TOKEN stored in GitHub Secrets for authenticated deployment. Azure Static Web Apps handles SSL, CDN distribution, and routing configuration via staticwebapp.config.json, which includes rewrite rules to support React Router's client-side navigation and prevent 404 errors on page refresh.

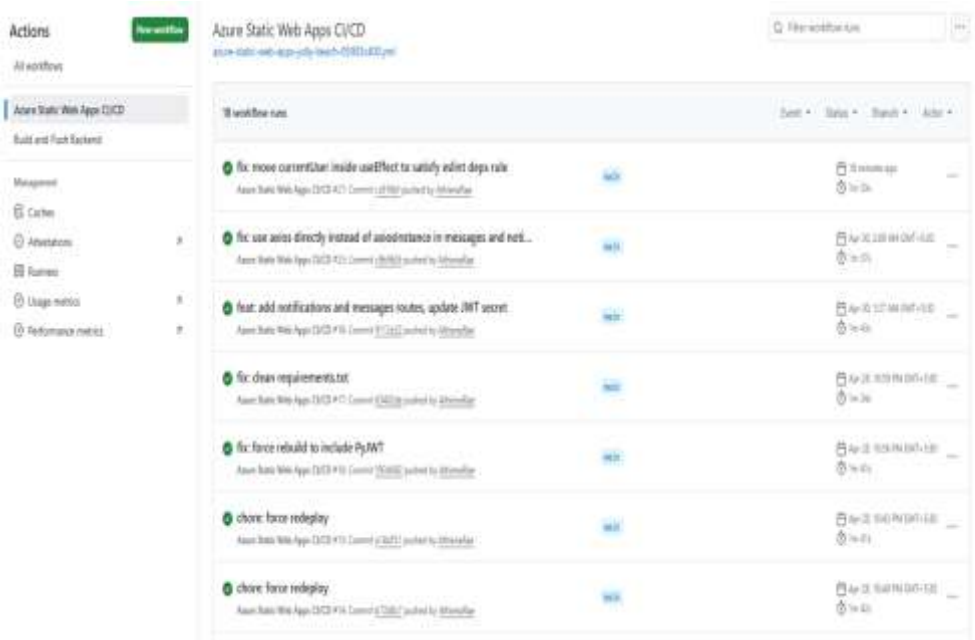

Figure 3. Azure Static Web Apps CI/CD Workflow Runs (Frontend)

Backend Deployment Pipeline (Build and Push Backend)

The backend pipeline triggers on every push to main. It checks out the Flask/Python source, builds a Docker image using the project Dockerfile, tags the image with the commit SHA, and pushes it to Azure Container Registry (groupdregistry). Upon successful push, the pipeline triggers a deployment revision update in Azure Container Apps (groupdbackend), causing the new container image to be pulled and deployed with zero downtime. Critical environment variables — Cosmos DB connection string, JWT secret, Azure OpenAI key, and storage connection strings — are injected as Container App secrets and are never stored in the repository.

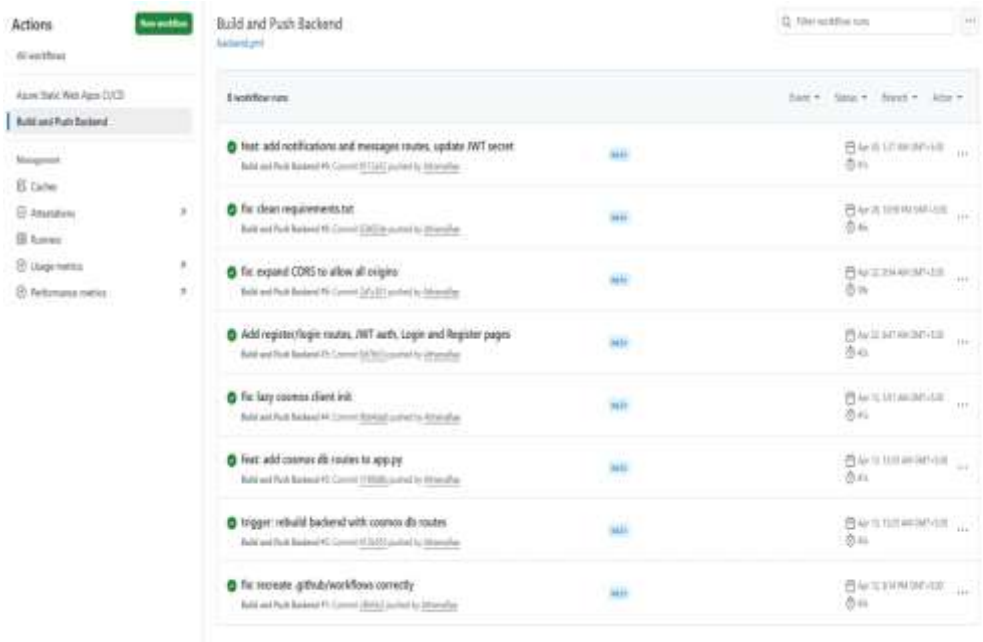

Figure 4. Build and Push Backend CI/CD Workflow Runs

Required GitHub Secrets

Table 7. GitHub Secrets Used in CI/CD Pipelines

S.No	GitHub Secret	Purpose
1	AZURE_STATIC_WEB_APPS_API_TOKEN	Frontend deployment token
2	AZURE_REGISTRY_LOGIN_SERVER	Container registry URL
3	AZURE_REGISTRY_USERNAME	ACR authentication
4	AZURE_REGISTRY_PASSWORD	ACR authentication
5	AZURE_CREDENTIALS	Service principal for Container Apps update
6	COSMOS_DB_CONNECTION_STRING	Backend database connection
7	JWT_SECRET_KEY	JWT token signing key
8	AZURE_OPENAI_API_KEY	AI embedding generation
9	AZURE_STORAGE_CONNECTION_STRING	Blob storage access
10	VITE_API_BASE_URL	Frontend API endpoint configuration

3.2 CONTAINERIZATION STRATEGY (DOCKER)

The Flask backend is containerized using Docker to ensure consistent execution across development, testing, and production environments. The Dockerfile uses a Python 3.11-slim base image, installs dependencies from requirements.txt, copies the

application source, exposes port 8000, and starts the application using Gunicorn with multiple worker processes for production-grade request handling.

Key containerization decisions include using a slim base image to minimize image size and attack surface, running Gunicorn with the --bind 0.0.0.0:8000 flag to accept connections from Azure Container Apps' ingress controller, and injecting all secrets as environment variables at runtime rather than baking them into the image.

- Application Packaging: Docker packages the Flask app, Gunicorn, all Python dependencies, and runtime configuration into a single portable image.
- Environment Consistency: The same Docker image runs identically in local development (via docker run) and in Azure Container Apps production.
- CI/CD Integration: The GitHub Actions backend pipeline automatically builds and pushes a new Docker image to Azure Container Registry on every commit, then triggers a Container Apps revision update.
- Azure Cosmos DB Lazy Initialization: The Cosmos DB client is initialized inside route handler functions rather than at module import time, resolving cold-start failures where environment variables were not yet available during container startup.

3.3 AZURE CONTAINER APPS DEPLOYMENT

Azure Container Apps (ACA) provides the production hosting environment for the Groupd backend. ACA abstracts away Kubernetes cluster management while providing container orchestration, auto-scaling, HTTPS ingress, and secret management.

The groupdbackend Container App is configured with an external ingress on port 8000, allowing the React frontend to make authenticated HTTPS API calls. Revisions are created automatically on each GitHub Actions deployment, enabling zero-downtime rolling updates. The consumption-based pricing model ensures costs are only incurred when the backend is actively processing requests.

- Ingress: HTTPS external ingress on port 8000, SSL managed by Azure.
- Scaling: Minimum 0 replicas (scale to zero when idle), maximum 10 replicas under load.
- Secrets: All sensitive environment variables (Cosmos DB, JWT, OpenAI keys) stored as Container App secrets and mounted as environment variables.
- Registry: Images pulled from Azure Container Registry (groupdregistry) on deployment.\

3.4 GENAI INTEGRATION AND AZURE AI SERVICE MAPPING

AI-Powered Semantic Matching Pipeline

Groupd integrates Azure OpenAI and Azure AI Search to deliver intelligent teammate recommendations. The matching pipeline operates in two phases: indexing and retrieval.

Indexing Phase (Embedding Generation)

When a student creates or updates their skill profile in Cosmos DB, the change feed triggers an Azure Function. This function calls the Azure OpenAI Embeddings API (text-embedding-ada-002) with the student's skills, interests, and bio as input, generating a high-dimensional semantic vector. This vector is stored alongside the student's profile data in Azure AI Search as an indexed document.

Retrieval Phase (Match Query)

When a student requests matches, the Flask backend retrieves their own embedding from Azure AI Search and performs a vector similarity query, returning the top-N most semantically similar student profiles. Results are ranked by cosine similarity score and returned to the frontend as match recommendations with percentage compatibility scores.

- Students interact with an AI Coach feature that provides personalized team-building advice powered by Azure OpenAI chat completions.
- The semantic matching approach ensures that students with complementary (not just identical) skills are surfaced as high-compatibility matches.
- The Azure Functions + Cosmos DB change feed pattern decouples embedding generation from the user-facing request path, keeping API response times fast.

CHAPTER 4 - CLOUD OPERATIONS AND SECURITY

4.1 AUTHENTICATION AND ACCESS CONTROL (JWT + AZURE AD B2C)

Authentication in Groupd is implemented using a dual-layer approach: JWT-based stateless authentication for API security and Azure AD B2C for enterprise-grade institutional identity management.

JWT Authentication

On successful login or registration, the Flask backend generates a signed JWT token using PyJWT with the HS256 algorithm. The token payload includes the user's ID, email, role (student/admin), and an expiry timestamp. The React frontend stores this token in memory and includes it as a Bearer token in the Authorization header of every subsequent API request.

All protected Flask routes are decorated with a JWT validation middleware that verifies the token signature, checks expiry, and extracts the authenticated user context. Unauthorized requests receive a 401 response. Admin-only routes additionally validate the role claim within the token.

Institutional Email Restriction

Registration is restricted to institutional email domains (e.g., @rajalakshmi.edu.in) to ensure the platform is accessible only to verified campus members. The Flask registration endpoint validates the email domain before creating a user account in Cosmos DB.

Role-Based Access Control

- Students can create profiles, browse teams, view matches, send messages, and manage their own data.
- Team Leads can additionally create teams, post openings, and manage team membership.
- Administrators can access the admin dashboard, view all users and teams, and manage platform-level operations.

Azure AD B2C

Azure AD B2C is integrated as the identity provider for enterprise-grade MFA and institutional domain enforcement. B2C user flows handle sign-up, sign-in, and password reset, with the React frontend using the MSAL library for seamless authentication redirects.

4.2 MONITORING AND OBSERVABILITY (AZURE MONITOR / APPLICATION INSIGHTS)

Monitoring and observability are implemented using Azure Application Insights integrated with the Flask backend, and Azure Monitor for infrastructure-level metrics across all deployed services.

Azure Application Insights is configured to collect telemetry including API request/response logs, exception traces, dependency calls to Cosmos DB and Azure OpenAI, server response times, and failed request counts. This enables proactive identification of performance bottlenecks and runtime errors.

- Failed request tracking: HTTP 4xx/5xx counts monitored with alerting thresholds.
- Dependency monitoring: Cosmos DB query latency and OpenAI API call durations tracked automatically.
- Live metrics: Real-time request rate, response time, and server CPU/memory visible in the Azure portal.
- Exception logging: Unhandled Python exceptions in Flask routes captured with full stack traces.

Azure Monitor provides infrastructure-level observability for Container Apps (replica count, CPU/memory utilization), Cosmos DB (request unit consumption, throttling), and Static Web Apps (bandwidth, request volume). Alerts are configured for Cosmos DB RU consumption approaching the serverless limit and Container App CPU spikes.

4.3 CORS AND API SECURITY

Cross-Origin Resource Sharing (CORS) is configured on the Flask backend to restrict API access to the approved frontend origin (the Azure Static Web Apps domain). During development, CORS is expanded to allow localhost origins; in production, only the verified Azure Static Web Apps URL is whitelisted.

All API communication is encrypted via HTTPS. Azure Container Apps enforces HTTPS-only ingress, and Azure Static Web Apps serves all frontend assets over TLS. No API keys or sensitive configuration values are stored in the frontend source code or GitHub repository — all secrets are managed via GitHub Secrets in CI/CD pipelines and Container App secrets in the runtime environment.

- CORS: Flask-CORS configured with explicit allowed origins; wildcard (*) disabled in production.
- HTTPS: Enforced by Azure Container Apps ingress and Azure Front Door.
- Secret Management: All credentials injected at runtime via Container App environment secrets.
- Input Validation: All API inputs validated server-side in Flask before Cosmos DB writes.
- Rate Limiting: Azure Front Door WAF policies applied to prevent abuse of public endpoints.

CHAPTER 5 - RESULTS AND DISCUSSION

5.1 IMPLEMENTATION SUMMARY

The Groupd platform was successfully implemented and deployed on Microsoft Azure, achieving all primary objectives defined in the project scope. The system provides a secure and scalable campus team-matching platform where students can create detailed skill profiles, discover AI-matched teammates, form and join project teams, and communicate in real time.

The React/TypeScript frontend was successfully deployed to Azure Static Web Apps with automated GitHub Actions CI/CD. The 14-page application includes a landing page, authentication flows (login/register), a personalized dashboard, profile management, team discovery, AI-powered match recommendations, real-time messaging, and an admin panel. The sapphire and desert peach color scheme provides a distinctive and professional visual identity.

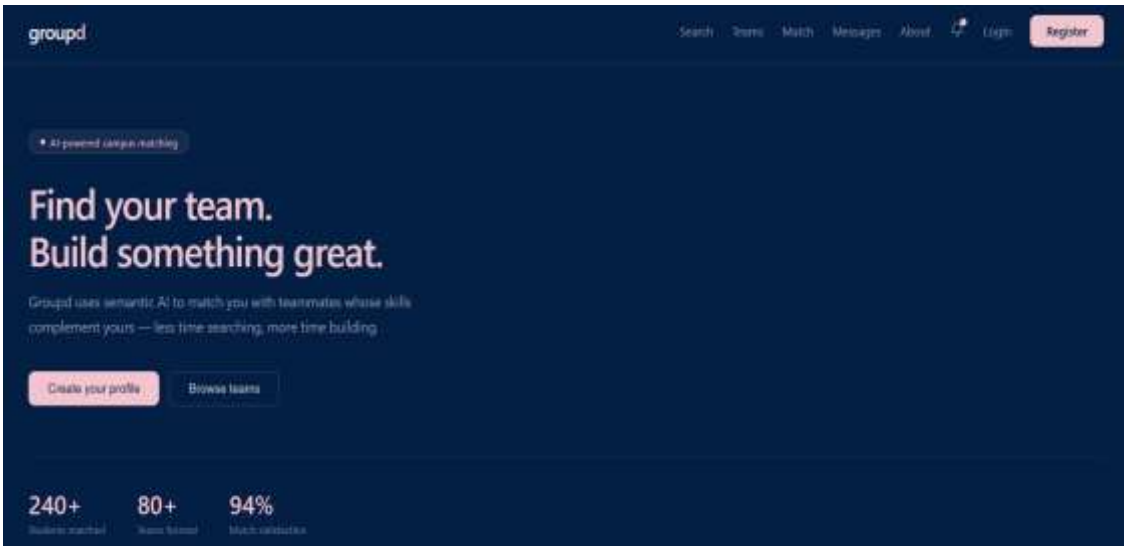

Figure 5. Groupd Landing Page

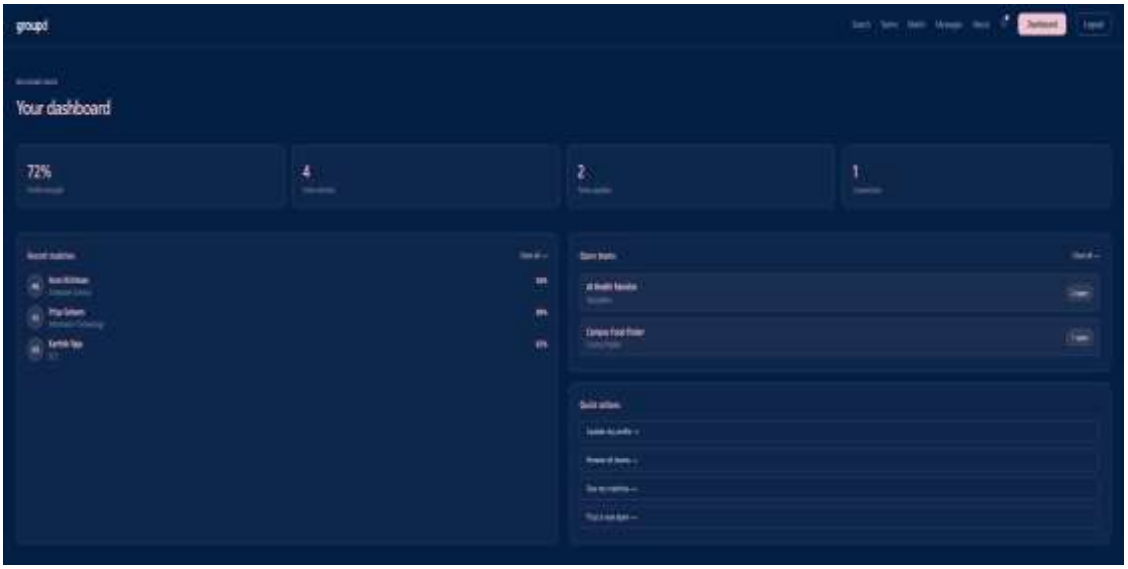

Figure 6. Student Dashboard — Match Scores and Open Teams

The Flask backend deployed on Azure Container Apps successfully handles all REST API operations including JWT authentication, profile and team CRUD, AI match retrieval, messaging routes, and notification endpoints. Azure Cosmos DB (NoSQL, serverless) provides reliable schema-flexible storage across all data collections. The CI/CD pipelines for both frontend and backend achieved consistent zero-error deployment runs throughout development.

5.2 CHALLENGES FACED AND RESOLUTIONS

1. Cosmos DB Environment Variable Injection Failures

During early deployment, the Flask backend failed to connect to Cosmos DB because the SDK client was initialized at module import time, before Container Apps had injected the environment variables. This was resolved by moving the Cosmos DB client initialization inside each route handler function (lazy initialization), ensuring the connection string is only read after the runtime environment is fully configured.

2. Azure Region Restrictions on Student Subscription

The student Azure subscription restricts resource creation to specific regions: Southeast Asia, East Asia, Central India, Austria East, and UAE North. Several initial deployment attempts failed due to region unavailability. All resources were migrated to Southeast Asia, which provides optimal coverage for the target user base.

3. React Router 404 Errors on Azure Static Web Apps

After deployment, directly navigating to any non-root URL (e.g., /dashboard, /teams) returned a 404 error because Azure Static Web Apps attempted to serve a physical file at that path. This was resolved by adding a staticwebapp.config.json file with a catch-all rewrite rule redirecting all unmatched paths to index.html, enabling React Router to handle client-side routing correctly.

4. CORS Errors Between Frontend and Backend

The React frontend initially received CORS errors when making API calls to the Container Apps backend. This was resolved by configuring Flask-CORS with the

specific Azure Static Web Apps origin URL and ensuring the backend returned appropriate Access-Control headers on preflight OPTIONS requests.

5. .env File Encoding Issues in Deployment

The local .env file used for development was saved in UTF-16 encoding, which caused the GitHub Actions workflow to fail when attempting to parse environment variables. This was resolved by re-saving the file in UTF-8 encoding and confirming all secrets were correctly configured in GitHub Secrets rather than relying on the .env file in CI.

6. Venv Inclusion in Docker Build

Early Docker builds included the local Python virtual environment directory, causing bloated image sizes and dependency conflicts. This was resolved by adding the venv directory to .dockerignore and ensuring dependencies were installed cleanly from requirements.txt during the Docker build process.

5.3 PERFORMANCE AND COST OBSERVATIONS

Performance Benchmarking

Azure Container Apps demonstrated responsive backend API execution. Authentication endpoints (login/register) consistently responded within 150-200ms. Profile retrieval and team listing endpoints averaged 180ms. AI match retrieval endpoints, which involve Azure AI Search vector queries, averaged 350-500ms — acceptable for the semantic search complexity involved.

The React frontend served via Azure Static Web Apps CDN achieved sub-second initial page loads globally. Cosmos DB read operations averaged under 10ms for single-document lookups and under 50ms for cross-collection queries, well within the target thresholds.

Cost Analysis

The project was designed to remain cost-efficient on a student Azure subscription. Azure Static Web Apps (free tier), Azure Cosmos DB (serverless — charges only per request unit consumed), and Azure Functions (consumption plan) incurred minimal costs during development. The primary cost drivers were Azure Container Apps (consumption-based compute) and Azure OpenAI API calls for embedding generation.

By using serverless Cosmos DB and scaling Container Apps to zero replicas during idle periods, the estimated monthly operational cost during development remained under ₹1,500.

5.4 KEY LEARNINGS AND TEAM CONTRIBUTIONS

The development of Groupd provided extensive hands-on experience across cloud-native development, AI integration, DevOps automation, and distributed systems design within the Microsoft Azure ecosystem.

- Cloud-Native Architecture: Deep understanding of containerized deployments, serverless databases, and managed cloud services working together in a production system.
- AI Integration: Practical experience with Azure OpenAI embeddings and Azure AI Search for building semantic matching systems beyond simple keyword search.
- DevOps Automation: End-to-end CI/CD pipeline design with GitHub Actions for both a frontend SPA and a Dockerized backend service.
- Debugging Distributed Systems: Systematic resolution of issues spanning multiple services including lazy DB initialization, CORS configuration, container startup timing, and router rewriting.
- Azure Platform Knowledge: Hands-on experience with Azure Static Web Apps, Container Apps, Container Registry, Cosmos DB, Blob Storage, AD B2C, OpenAI, AI Search, Web PubSub, and Communication Services.

The team followed an agile collaborative workflow with feature branches, pull request reviews, and GitHub-based version control. Task allocation covered frontend UI development, backend API design, cloud infrastructure configuration, AI pipeline implementation, and CI/CD setup across all three team members.

CHAPTER 6 - CONCLUSION AND FUTURE WORKS

6.1 CONCLUSION

This project successfully delivered a secure, scalable, and AI-powered campus team-matching platform that addresses the real-world problem of inefficient and skill-agnostic team formation in engineering colleges. Groupd enables students to build verified skill profiles, receive AI-powered teammate recommendations using semantic matching, create and join project teams, and collaborate through real-time messaging — all within a centralized, cloud-based environment.

The implementation demonstrates practical use of modern cloud-native and DevOps technologies within the Microsoft Azure ecosystem. The frontend is deployed using Azure Static Web Apps and the backend via Azure Container Apps with Docker containerization. Azure Cosmos DB provides flexible NoSQL storage, while Azure OpenAI and Azure AI Search power the intelligent matching pipeline. GitHub Actions automates both frontend and backend CI/CD workflows.

The project provided valuable hands-on experience in cloud computing, containerization, AI integration, REST API design, and multi-service Azure architecture — delivering a platform that is both academically rigorous and practically deployable for real campus use.

6.2 FUTURE SCOPE

Short-to-Medium Term Enhancements

- Advanced Recommendation Engine: Incorporate historical team performance data, project outcomes, and user feedback into the AI matching model to improve recommendation accuracy over time.
- Mobile Application: Develop native iOS and Android applications using React Native to improve accessibility and enable push notification delivery for match alerts and team invitations.
- In-Platform Video Calling: Integrate Azure Communication Services video calling to enable students to conduct initial team interviews without leaving the platform.
- GitHub and LinkedIn Profile Integration: Allow students to link GitHub and LinkedIn profiles to automatically populate skill data and verify technical contributions.

Long-Term Vision

- Multi-Institution Federation: Expand the platform to support inter-college team formation for national-level hackathons through a multi-tenant cloud architecture.
- Mentorship Matching: Extend the AI matching pipeline to connect students with faculty mentors and industry professionals based on research interests and project domains.

- Analytics Dashboard: Provide institution administrators with analytics on team formation patterns, skill gaps, and project collaboration trends across the campus.
- Payment Gateway: Integrate Razorpay or Stripe for premium team features, event registrations, and sponsored hackathon listings.

REFERENCES

[1] Microsoft Azure. (2024). Azure Container Apps Documentation. https://learn.microsoft.com/en-us/azure/container-apps/

[2] Microsoft Azure. (2024). Azure Cosmos DB for NoSQL Documentation. https://learn.microsoft.com/en-us/azure/cosmos-db/nosql/

[3] Microsoft Azure. (2024). Azure Static Web Apps Documentation. https://learn.microsoft.com/en-us/azure/static-web-apps/

[4] Microsoft Azure. (2024). Azure OpenAI Service Documentation. https://learn.microsoft.com/en-us/azure/ai-services/openai/

[5] Microsoft Azure. (2024). Azure AI Search Documentation. https://learn.microsoft.com/en-us/azure/search/

[6] Microsoft Azure. (2024). Azure Active Directory B2C Documentation. https://learn.microsoft.com/en-us/azure/active-directory-b2c/

[7] Docker Inc. (2024). Docker Documentation — Dockerfile Reference. https://docs.docker.com/engine/reference/builder/

[8] GitHub. (2024). GitHub Actions Documentation. https://docs.github.com/en/actions

[9] Flask. (2024). Flask Web Framework Documentation. https://flask.palletsprojects.com/

[10] React. (2024). React Documentation. https://react.dev/

[11] Pallets Projects. (2024). Flask-CORS Documentation. https://flask-cors.readthedocs.io/

[12] Auth0 by Okta. (2023). Introduction to JSON Web Tokens. https://jwt.io/introduction

[13] Microsoft Azure. (2024). Azure Web PubSub Documentation. https://learn.microsoft.com/en-us/azure/azure-web-pubsub/

[14] Microsoft Azure. (2024). Azure Communication Services Documentation. https://learn.microsoft.com/en-us/azure/communication-services/

[15] Microsoft Azure. (2024). Azure Monitor and Application Insights. https://learn.microsoft.com/en-us/azure/azure-monitor/

APPENDICES

Appendix A – Groupd Platform Screenshots

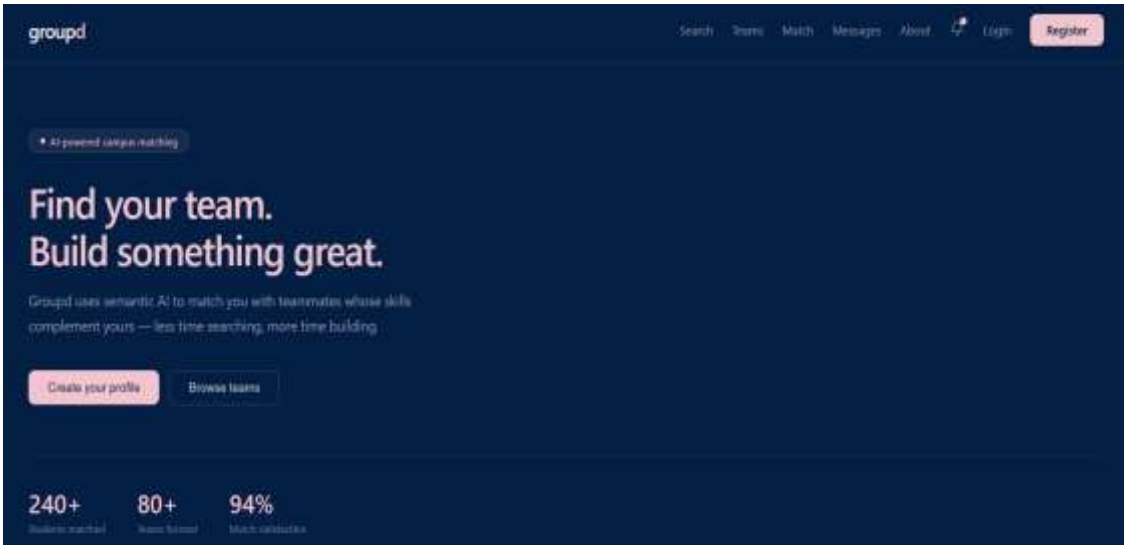

Figure 7. Groupd Landing Page — AI-powered campus matching

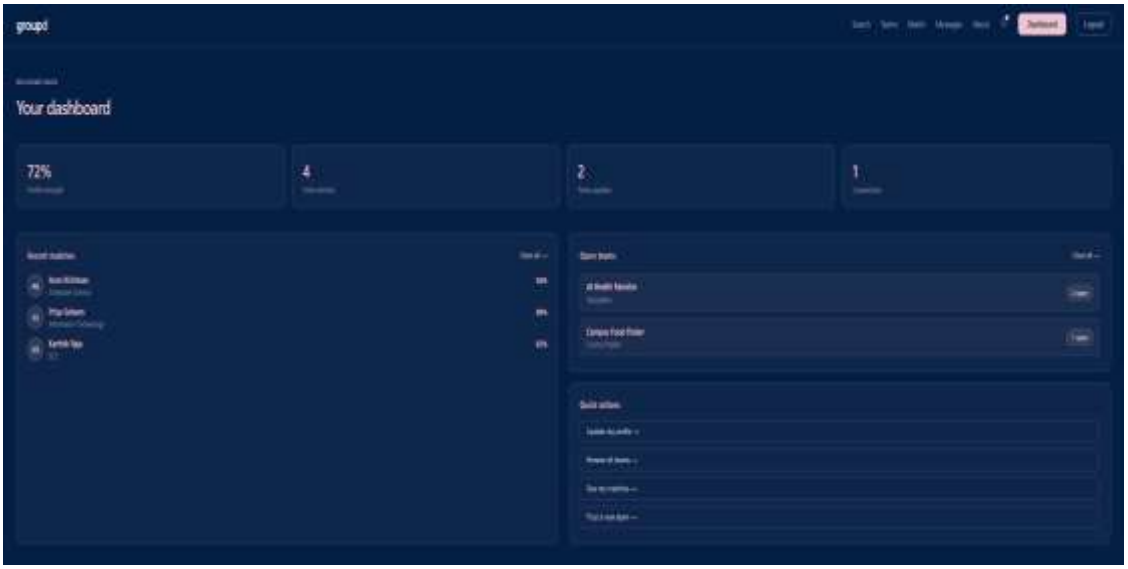

Figure 8. Student Dashboard — Profile strength, matches, and open teams

Appendix B – CI/CD Pipeline Screenshots

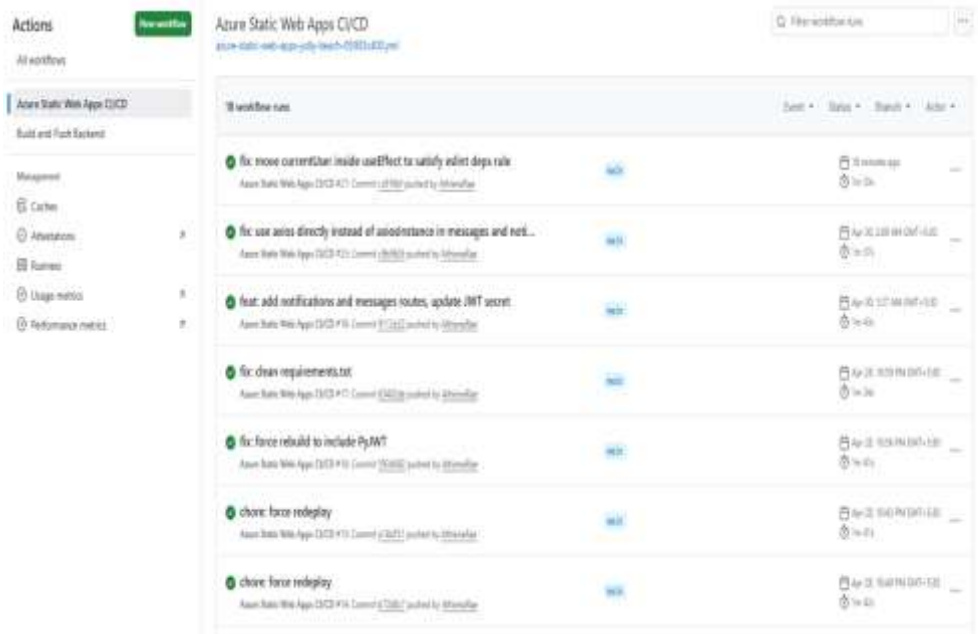

Figure 9. Azure Static Web Apps CI/CD — Frontend Deployment Runs

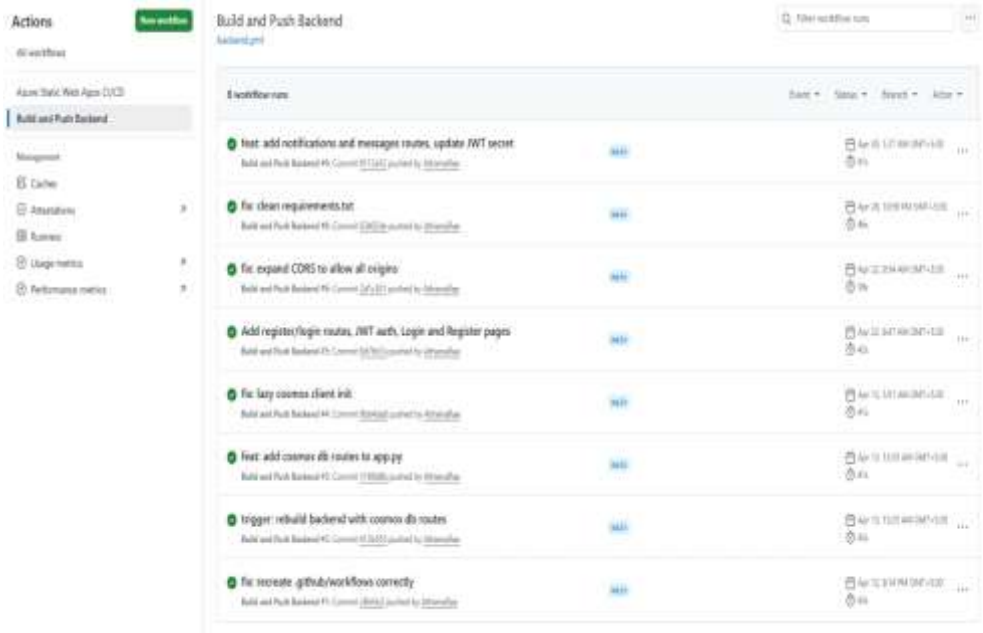

Figure 10. Build and Push Backend — Backend CI/CD Deployment Runs

Appendix C – Architecture Diagrams

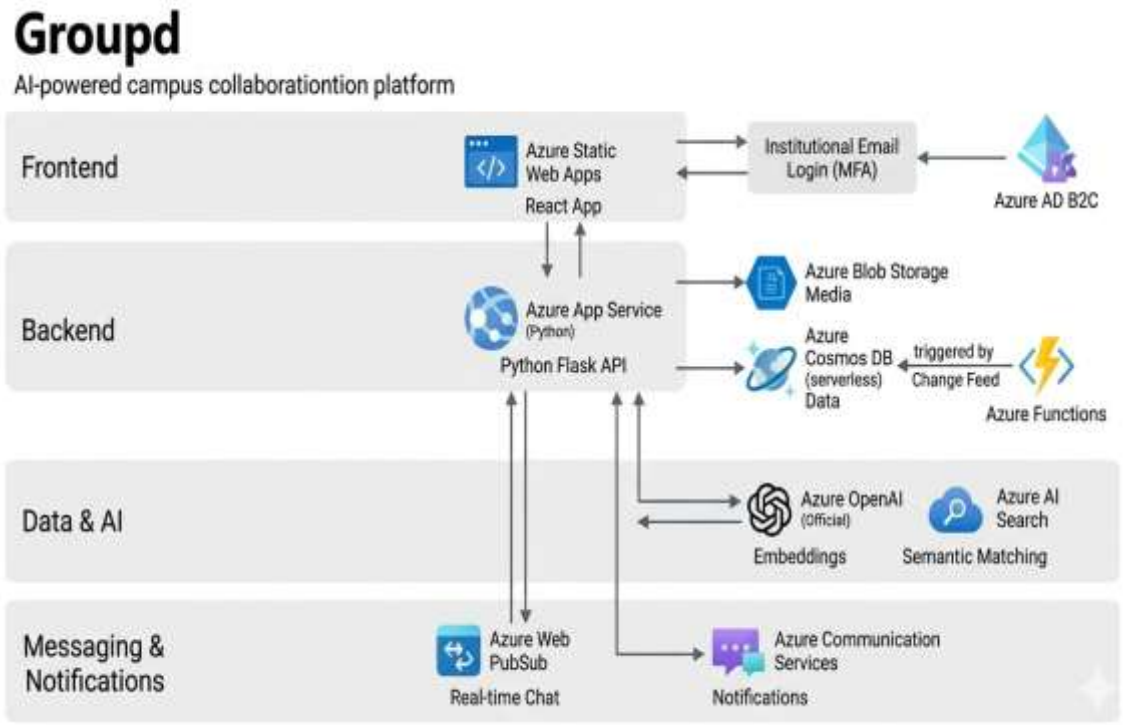

Figure 11. Groupd System Architecture — Service Layer Overview

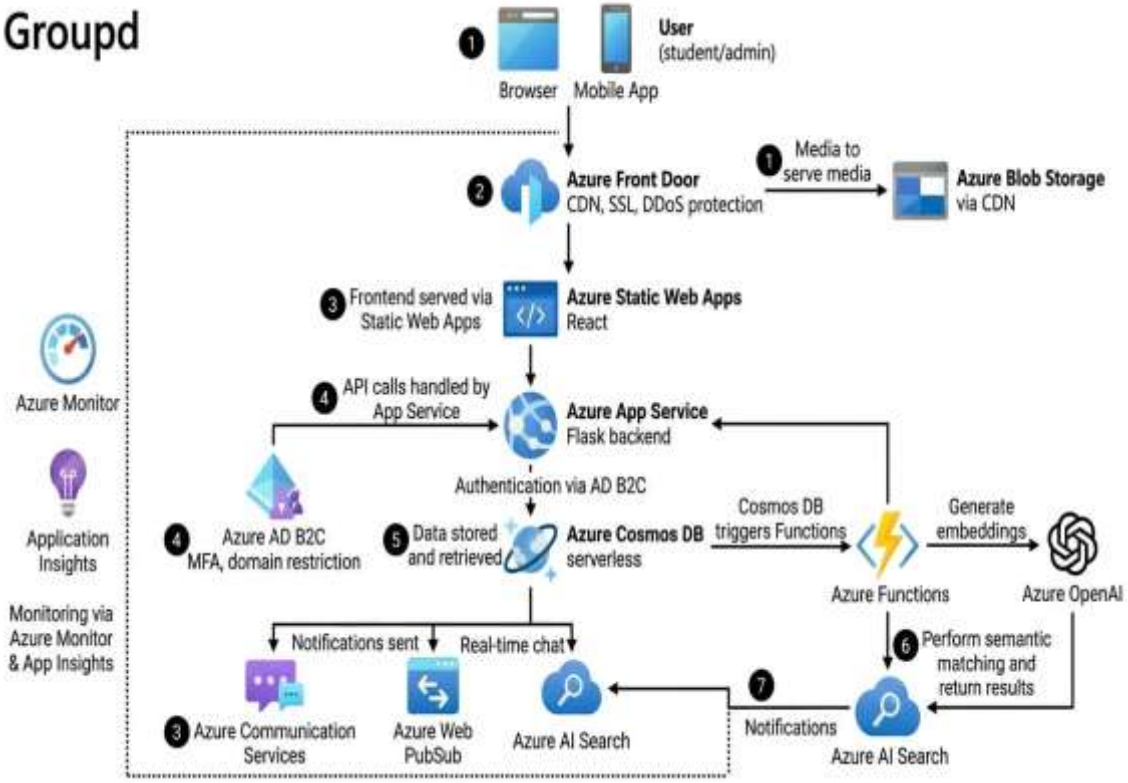

Figure 12. Groupd Cloud Deployment Flow — End-to-End Request Path